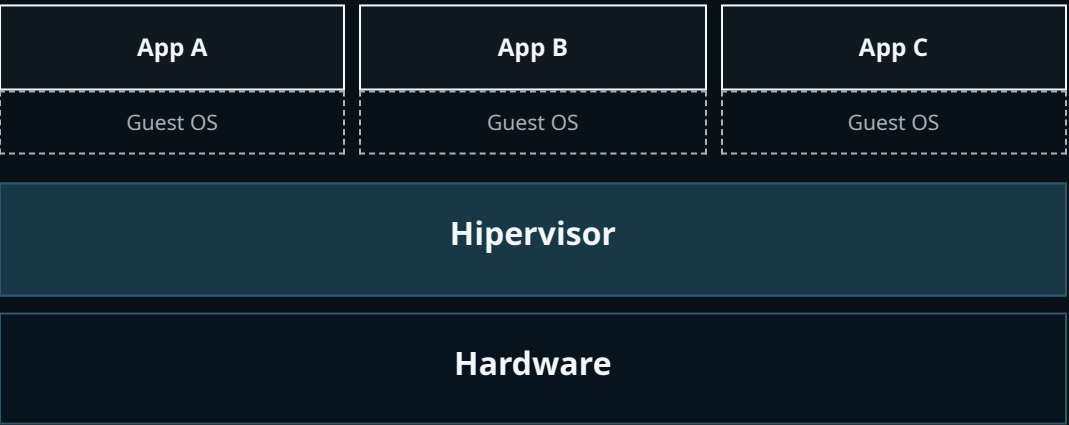


AULA 1

Containers, VMs e Docker

Do isolamento de ambientes até uma aplicação empacotada e executada da mesma forma em qualquer máquina.

MÁQUINAS VIRTUAIS



DOCKER E CONTAINERS



VIRTUAL MACHINE

VS

DOCKER ENGINE

OBJETIVO / AMBIENTE REPRODUZÍVEL

■ MANIFEST LOADED

Objetivo da aula

Entender o isolamento e empacotar uma aplicação sem depender da configuração manual de cada computador.

Vamos comparar bare metal, máquinas virtuais e containers; depois conectar Dockerfile, imagem, container, porta, volume e Docker Compose.

ANTERIOR

02 / 20

PRÓXIMO

VIRTUAL MACHINE

VS

DOCKER ENGINE

CONTEXTO / EVOLUÇÃO DAS APLICAÇÕES

■ CLOUD NATIVE

Da TI tradicional para aplicações nativas em nuvem

TI tradicional

Aplicações monolíticas, deploy lento, escala vertical e manutenção mais complexa.

TI nativa em nuvem

Microserviços, deploy automatizado, escala horizontal e ambientes reproduzíveis.

Containers ajudam nessa mudança porque empacotam a aplicação e suas dependências, reduzindo diferenças entre desenvolvimento, teste e produção.

ANTERIOR

03 / 20

PRÓXIMO

VIRTUAL MACHINE

VS

DOCKER ENGINE

BARE METAL / MÁQUINA FÍSICA

DIRECT HARDWARE

Aplicações

Sistema operacional

Hardware físico

A aplicação usa o sistema operacional instalado diretamente no hardware. Há poucas camadas, mas os ambientes ficam presos à configuração daquela máquina.

ANTERIOR

04 / 20

PRÓXIMO

VIRTUAL MACHINE

DOCKER ENGINE

VS

VIRTUALIZAÇÃO / MÁQUINAS COMPLETAS

■ HYPERVISOR ACTIVE

Máquinas virtuais

App A

App B

Guest OS

Guest OS

App C

Guest OS

Hipervisor

Hardware



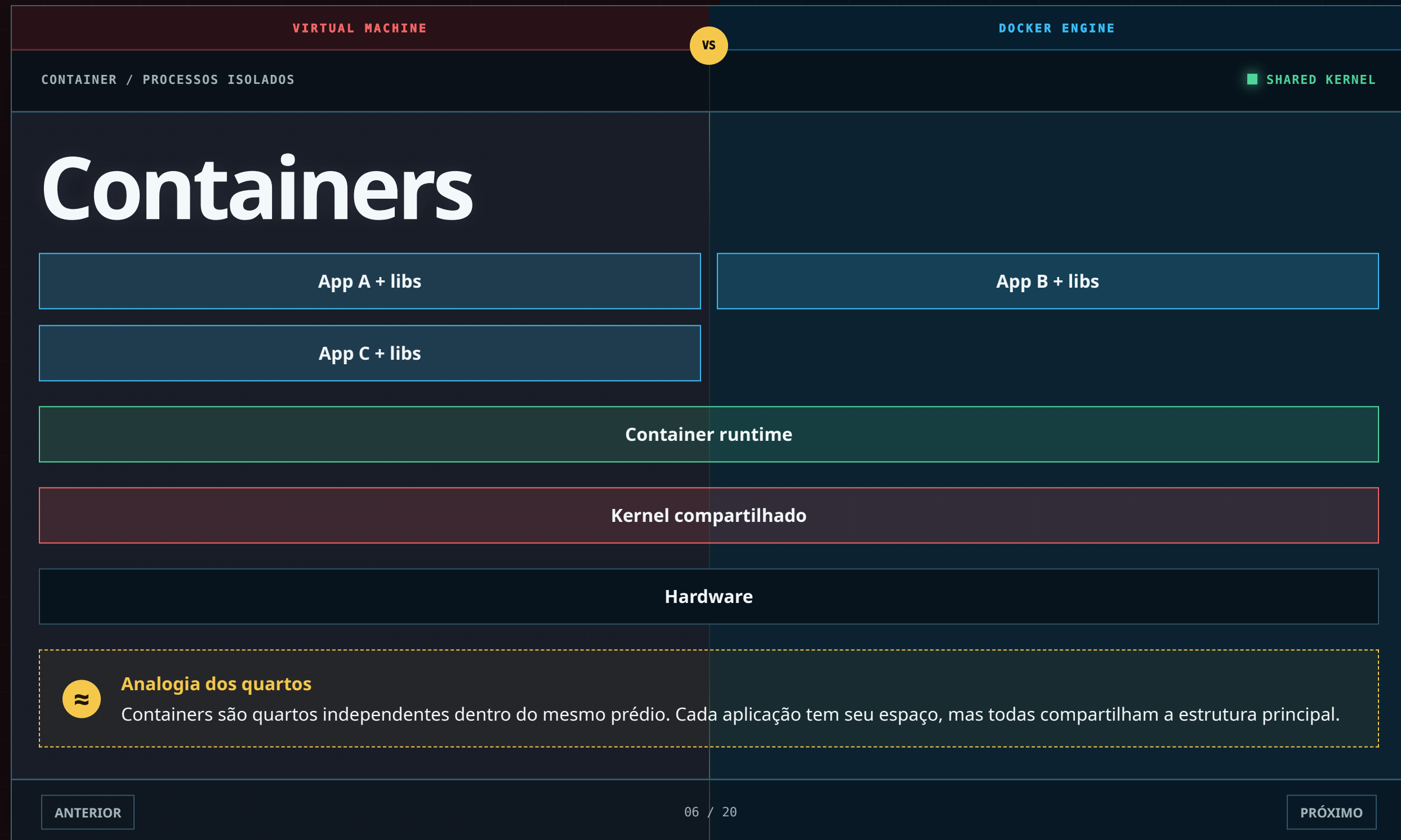
Analogia dos apartamentos

Cada VM é um apartamento completo: tem seu próprio sistema, instalações e isolamento. Isso oferece independência, mas ocupa mais espaço e recursos.

ANTERIOR

05 / 20

PRÓXIMO



VIRTUAL MACHINE

DOCKER ENGINE

VS

COMPARAÇÃO / VM VERSUS CONTAINER

■ TRADE-OFFS VISIBLE

VM e container não são a mesma coisa

Máquina virtual

Sistema operacional completo, tamanho maior, inicialização mais lenta e isolamento mais forte.

Container

Processo isolado, kernel compartilhado, imagem menor e inicialização rápida.

Container não é uma “VM pequena”. A diferença principal está no que cada abordagem virtualiza e compartilha.

ANTERIOR

07 / 20

PRÓXIMO

VIRTUAL MACHINE

VS

DOCKER ENGINE

DECISÃO / ESCOLHER ISOLAMENTO

■ USE CASE SELECTED

Quando usar cada um?

Containers

APIs, microsserviços, pipelines, ambientes locais reproduzíveis e escala horizontal.

VMs

Sistemas operacionais diferentes, aplicações legadas e isolamento de máquina completa.

As abordagens também podem coexistir: containers frequentemente rodam dentro de VMs em provedores de nuvem.

ANTERIOR

08 / 20

PRÓXIMO

VIRTUAL MACHINE

VS

DOCKER ENGINE

DOCKER / PLATAFORMA

■ DAEMON ONLINE

Onde entra o Docker?

Construir

Cria imagens a partir de instruções.

Distribuir

Publica e baixa imagens de registries.

Docker é uma plataforma que facilita trabalhar com containers. Container é o conceito; Docker é uma das ferramentas usadas para criá-los e executá-los.

Executar

Inicia e gerencia containers.

ANTERIOR

09 / 20

PRÓXIMO

VIRTUAL MACHINE

VS

DOCKER ENGINE

FLUXO / BUILD AND RUN

■ PIPELINE READY

Do arquivo à aplicação rodando

Dockerfile

instruções

docker build

construção

Imagem

template

docker run

container



Analogia da fábrica

O Dockerfile é o projeto de fabricação, a imagem é o produto embalado e o container é uma unidade aberta e funcionando.

ANTERIOR

10 / 20

PRÓXIMO

VIRTUAL MACHINE

VS

DOCKER ENGINE

DOCKERFILE / RECEITA

INSTRUCTIONS PARSED

Dockerfile

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

A imagem base já contém o servidor web. O arquivo da aplicação é copiado para o diretório publicado pelo Nginx, e a porta 80 documenta onde o serviço atende.

ANTERIOR

11 / 20

PRÓXIMO

VIRTUAL MACHINE

DOCKER ENGINE

VS

IMAGEM / TEMPLATE IMUTÁVEL

■ TAG V1 READY

Imagem Docker

Camada: index.html

Camada: servidor Nginx

Camadas da imagem base

Imutável

Uma versão construída não muda.

Versionada

Tags identificam versões.

Reutilizável

Cria vários containers iguais.

ANTERIOR

12 / 20

PRÓXIMO

CONTAINER / INSTÂNCIA

■ RUNNING

Container é a imagem em execução

api-1

imagem minha-api:v1

ISO / APP

api-3

imagem minha-api:v1

ISO / APP

api-2

imagem minha-api:v1

ISO / APP

api-4

imagem minha-api:v1

ISO / APP

Os containers são processos separados criados a partir do mesmo template. Parar um container não apaga a imagem usada para criá-lo.

VIRTUAL MACHINE

VS

DOCKER ENGINE

REDE / PUBLICAÇÃO DE PORTA

■ 8080 EXPOSED

Porta do host e porta do container

```
docker run -p 8080:8080 minha-api:v1
```

```
localhost:8080 → container:8080
```



Analogia do endereço e apartamento

A porta do host é a entrada do prédio. A porta interna identifica onde a aplicação atende dentro do container.

• `EXPOSE 8080` documenta a porta. • `-p 8080:8080` publica essa porta para acesso pelo host.

ANTERIOR

14 / 20

PRÓXIMO

VIRTUAL MACHINE

DOCKER ENGINE

VS

ARQUITETURA / DOCKER ENGINE

■ DAEMON RUNNING

Docker Engine: CLI, API e Daemon

Docker CLI
docker ...

Docker API
requisições

Docker Daemon
dockerd

Imagens e
containers

A CLI recebe o comando. A API leva a solicitação ao daemon, que cria e gerencia imagens, containers, redes e volumes.

ANTERIOR

15 / 20

PRÓXIMO

VIRTUAL MACHINE

DOCKER ENGINE

VS

REGISTRY / DISTRIBUIÇÃO DE IMAGENS

■ IMAGE AVAILABLE

Docker Registry

Docker Hub

Registry público com imagens prontas, como nginx, postgres e mongo.

Registry privado

Armazena imagens internas com controle de acesso, como AWS ECR e Azure ACR.

⌘ `docker pull` baixa uma imagem. Depois de autenticar e versionar corretamente, ⌘ `docker push` publica uma imagem no registry.

ANTERIOR

16 / 20

PRÓXIMO

VIRTUAL MACHINE

DOCKER ENGINE

VS

COMPOSE / VÁRIOS SERVIÇOS

■ STACK DEFINED

Docker Compose

Serviços

Define os containers que fazem parte da aplicação.

Volumes

Define onde dados persistentes serão armazenados.

Redes

Define como os serviços se comunicam.

```
docker-compose up -d
docker-compose logs
docker-compose down
```

O arquivo `docker-compose.yml` permite iniciar e encerrar vários containers com poucos comandos.

ANTERIOR

17 / 20

PRÓXIMO

VIRTUAL MACHINE

VS

DOCKER ENGINE

COMANDOS / OPERAÇÃO BÁSICA

■ TERMINAL READY

Comandos que você realmente usa

```
docker pull nginx:alpine
docker images
docker build -t site-docker:v1 .
docker run -d -p 8080:80 \
  --name site-docker site-docker:v1
docker ps
docker logs site-docker
```

```
docker stop site-docker
docker start site-docker
docker exec -it site-docker sh
docker rm site-docker
```

```
docker stats
docker image prune
```

Esses comandos cobrem download, build, execução, inspeção, logs, parada e manutenção básica.

[ANTERIOR](#)

18 / 20

[PRÓXIMO](#)

VIRTUAL MACHINE

VS

DOCKER ENGINE

PRÁTICA / SERVIDOR WEB

■ SITE RUNNING

Prática: página web em um container

```
<h1>
  Aplicação executando
  com Docker
</h1>
```

```
localhost:8080
    ↓
container:80
    ↓
index.html
```

```
docker build -t site-docker:v1 .
docker run -d -p 8080:80 --name site-docker site-docker:v1
```

O exercício usa somente HTML e Nginx para concentrar a aula nos conceitos de imagem, container, build e publicação de portas.

ANTERIOR

19 / 20

PRÓXIMO

VIRTUAL MACHINE

VS

DOCKER ENGINE

RESUMO / CARGA ENTREGUE

COMPLETE

Resumo

VM

Virtualiza uma máquina completa.

Imagem

Template imutável e versionado.

Docker não elimina a infraestrutura. Ele transforma o ambiente da aplicação em algo descrito, reproduzível e mais fácil de transportar.

ANTERIOR

20 / 20

INÍCIO

Container

Isola processos compartilhando o kernel.

Compose

Descreve vários containers juntos.